

CLOUD SECURITY & DEVSECOPS

Final Case Study Report

SecureFlow DevSecOps Pipeline Engagement

Detecting, Remediating, and Enforcing Security Across a Banking Microservices Platform

Field	Detail
Analyst	Omajuwa Okotie
Target System	SecureFlow, banking microservices platform
Engagement	DevSecOps Pipeline Engagement, Two-Week Programme (10 working days)
Environment	Local kind Kubernetes cluster; AWS (Terraform-defined, unapplied)
Technology Stack	Kubernetes, Terraform, Docker, HashiCorp Vault, OPA Gatekeeper, Falco
Security Scanning	Gitleaks, SonarQube, Trivy, Checkov, OWASP ZAP
CI/CD	GitHub Actions, 6-job pipeline + differentiated Security Gate
Repository	github.com/omjhub/SecureFlow-VulnerableCodebase
Date	August – September 2026

Prepared by: Omajuwa Okotie

Table of Contents

1. Executive Summary
2. Project Context
3. Solution Architecture
4. Week 1 Implementation: Detection Pipeline
5. Week 2 Implementation: Remediation, Enforcement, Proof
6. Security Assessment: Before / After
7. CI/CD Pipeline and the Security Gate
8. Runtime Security: Falco Evidence
9. Secrets Management: Vault
10. Dynamic Application Security Testing
11. Observability: Grafana Dashboard
12. Results Against Success Metrics
13. Challenges and Lessons Learned
14. Production Gaps and Future Enhancements
15. Conclusion
16. Appendix: Supplementary Evidence

1. Executive Summary

This report documents a security engagement carried out against SecureFlow, a three-service banking platform, covering detection, remediation, and enforcement across the full stack over a two-week programme. The objective was to build a working system, not a one-off remediation pass: an automated CI/CD pipeline that finds every vulnerability class on every push, a differentiated Security Gate that blocks merges on infrastructure-owned findings while routing application-layer findings to an AppSec intake process without blocking delivery, and a hardened, monitored runtime environment.

The engagement followed a detect → categorise → remediate → verify discipline, applied consistently across every layer: application code, container images, Kubernetes configuration, cloud infrastructure, secrets management, and live runtime behaviour. Every fix was verified using the same tool that found the original issue, never assumed from the change alone.

Checkov (Terraform IaC) was brought from 72 failed checks to 0. Trivy image, Trivy Kubernetes config, and Gitleaks all reached 0 blocking findings. The Security Gate's final verified result across the entire pipeline: PASSED, no DevSecOps-owned findings.

Two unplanned infrastructure incidents occurred mid-engagement, a complete, unexplained loss of the local Kubernetes cluster, and a Docker Desktop resource-exhaustion event that also revealed a missing Grafana persistence volume. Both were treated as genuine tests of the project's own infrastructure-as-code discipline, and full recovery was achieved from committed manifests and documented procedure alone in both cases.

Authenticated dynamic testing (OWASP ZAP) independently rediscovered a SQL injection / authentication-bypass vulnerability first proven by manual exploitation in Week 1, and separately confirmed a live, reachable Werkzeug interactive debugger, a finding serious enough to elevate a previously-documented information-disclosure issue to a critical, remote-code-execution-adjacent severity.

This report reports findings honestly, including tool limitations, correctly-identified false positives, and deliberately scoped follow-up items, rather than presenting the engagement as uniformly and universally resolved.

2. Project Context

2.1 About SecureFlow

SecureFlow is a three-service banking platform, authentication, transaction processing, and a frontend, each backed by its own PostgreSQL database. A security assessment identified vulnerabilities across every layer of the stack (application code, container images, Kubernetes manifests, and Terraform infrastructure), classified and tracked to closure using an internal severity taxonomy (AV-*, TV-*, FV-*, IV-*, CK-* by category).

2.2 The Business Problem

A codebase and infrastructure definition with no automated security scanning, no differentiated ownership between infrastructure and application findings, no secrets management beyond plaintext environment variables, and no runtime detection presents the same risk profile as any real production system managed without these controls: vulnerabilities ship silently, credentials leak into version control, and a compromised container can act freely with no detection layer watching it.

2.3 Project Goal

To prove that a vulnerable codebase can be systematically hardened through an automated, auditable pipeline, not through one-off manual fixes, and to implement the ownership-aware gating discipline that distinguishes a professional DevSecOps programme from an ad hoc vulnerability sweep: findings owned by infrastructure engineering block delivery; findings owned by an application security team are surfaced and routed, never silently dropped, but also never allowed to block a deployment the requesting engineer has no way to fix themselves.

3. Solution Architecture

3.1 Why This Stack

Every tool selected is production-grade and in active, real-world use at organisations operating at scale, not a simplified teaching substitute. Kubernetes, Terraform, GitHub Actions, HashiCorp Vault, Falco, and OPA Gatekeeper are the same tools a real platform engineering team would reach for.

3.2 Architecture Overview

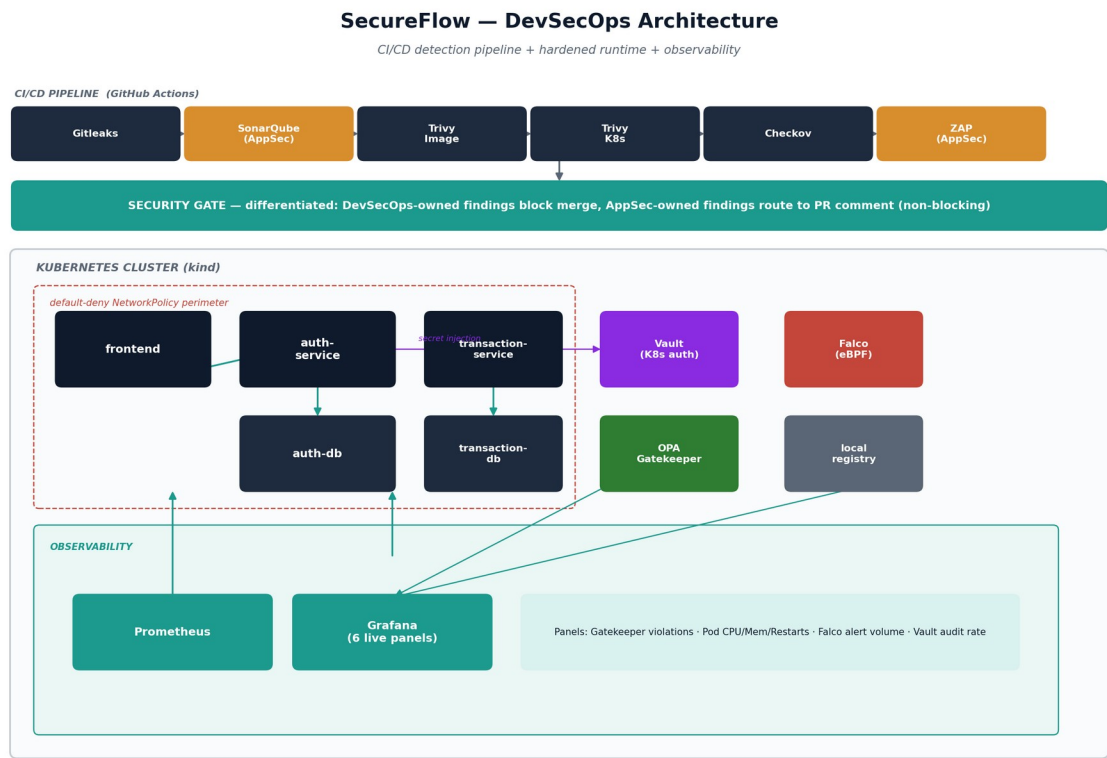


Figure 1: SecureFlow architecture, CI/CD detection pipeline, hardened Kubernetes runtime, and the observability layer.

3.3 Six Documented Architecture Decisions

Decision	Rationale
Kustomize over Helm	Simpler environment-overlay patching for this project's scale; no templating engine overhead.
Trivy as single-tool scanner	One tool covers image CVEs, Kubernetes config, and SBOM generation, fewer moving parts.
OPA Gatekeeper over Kyverno	Rego is transferable beyond Kubernetes, a more broadly useful skill investment.
Vault over native Kubernetes Secrets	Real encryption, real audit logging, and dynamic credentials, not just base64-encoded text in etcd.
Differentiated Security Gate	Hard-fail only on infrastructure findings; prevents alert fatigue and blocked deploys over findings the requesting engineer cannot fix.
Spot instances	60–70% compute cost reduction; acceptable where guaranteed uptime is not

Decision	Rationale
(dev/staging, Terraform-defined)	required.

4. Week 1 Implementation: Detection Pipeline

Week 1 built the detection half of the system: a working exploit chain proven by hand, followed by five automated scanners wired into a GitHub Actions pipeline, each stage capturing full raw findings rather than a bare pass/fail signal.

4.1 Manual Exploitation, Proving the Vulnerabilities Are Real

A manual security assessment was performed first, to establish ground truth before introducing automated tooling. The authentication bypass (`admin'--`) returned a genuine, valid administrator JWT directly from the running service:

```

omajuwakotie@MacBookAir SecureFlow-VulnerableCodebase % curl -i -X POST http://localhost:5000/login \
-H 'Content-Type: application/json' \
-d '{"username": "admin", "password": "anything"}'
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Thu, 30 Jul 2026 17:22:52 GMT
Content-Type: application/json
Content-Length: 197
Connection: close

{
  "role": "admin",
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWx0eXkiLCJpc2YubmFtZSI6ImFkbWUiLCJ1cm91bmFtZSI6ImFkbWUuIn0.P2Er_Pqs60tb2YQ3wRkZgCOjABjAS9TqtQwzqlsI",
  "user_id": 1
}
omajuwakotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 2: Manual SQL injection against auth-service's /login endpoint, the admin'-- payload bypasses password verification entirely and returns a real, valid JWT for the admin account.

4.2 Secrets and Application Code Scanning

Gitleaks was configured with custom rules for the application's specific secret patterns and scanned full git history, not just the current tree, essential, since deleting a secret from current code does not remove it from history.

Summary

All jobs

GitLeaks Secret Scan

Run details

Usage

Workflow file

Annotations

1 error and 1 warning

GitLeaks Secret Scan

failed 8 minutes ago in 7s

Set up job

Checkout repository (full history)

Install GitLeaks

Run GitLeaks (full history scan)

```

1  Run gitileaks detect \
2  gitileaks detect \
3  --source . \
4  --log-opts="--all" \
5  --config .gitileaks.toml \
6  --report-format json \
7  --report-path gitileaks-report.json \
8  --redact \
9  --exit-code 1
10 shell: /usr/bin/bash -e {0}
11
12  o
13  | \
14  | o
15  o ||
16  gitileaks
17
18 3:52PM INF 11 commits scanned.
19 3:52PM INF scanned ~58507 bytes (58.51 KB) in 193ms
20 3:52PM WARN Leaks found: 6
21 Error: Process completed with exit code 1.

```

Upload GitLeaks report

Post Checkout repository (full history)

Complete job

Figure 3: Gitleaks full-history scan output, confirming real committed secrets detected in the repository's git history.

SonarQube was integrated for static application security testing, surfacing application-layer findings (SQL injection, weak hashing, XSS) that are explicitly AppSec-owned per the project's ownership matrix, detected and reported, never remediated directly in this engagement.

```

json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
onejuwakotie@MacBookAir SecureFlow-VulnerableCodebase % cat sonar-findings-main.json | python3 -c "
import json, sys
data = json.load(sys.stdin)
print('Total: ', data['total'])
for i in data['issues']:
    print(f'({i["severity"]}:10) ({i["rule"]}:35) ({i["component"].split("::")[-1]}):({i.get("line","")})')"
```

Severity	Rule	Component	Line
MAJOR	githubactions:S6737	.github/workflows/devsecops-pipeline.yml:55	55
MAJOR	githubactions:S6506	.github/workflows/devsecops-pipeline.yml:20	20
MAJOR	kubernetes:S6596	infra/kubernetes/base/auth-service.yml:20	20
MAJOR	kubernetes:S6428	infra/kubernetes/base/auth-service.yml:40	40
MAJOR	kubernetes:S6430	infra/kubernetes/base/auth-service.yml:41	41
MINOR	kubernetes:S6332	infra/kubernetes/base/configmap.yml:13	13
MINOR	kubernetes:S6332	infra/kubernetes/base/configmap.yml:14	14
MAJOR	kubernetes:S6596	infra/kubernetes/base/frontend.yml:18	18
MAJOR	kubernetes:S6428	infra/kubernetes/base/frontend.yml:25	25
MAJOR	kubernetes:S6430	infra/kubernetes/base/frontend.yml:26	26
MAJOR	kubernetes:S6596	infra/kubernetes/base/transaction-service.yml:18	18
MAJOR	kubernetes:S6428	infra/kubernetes/base/transaction-service.yml:37	37
MAJOR	kubernetes:S6430	infra/kubernetes/base/transaction-service.yml:38	38
BLOCKER	terraform:S6302	infra/terraform/modules/iam/main.tf:52	52
MAJOR	terraform:S6364	infra/terraform/modules/rds/main.tf:35	35
BLOCKER	terraform:S8793	infra/terraform/modules/rds/main.tf:46	46
MAJOR	terraform:S6303	infra/terraform/modules/rds/main.tf:47	47
MAJOR	terraform:S6364	infra/terraform/modules/rds/main.tf:85	85
BLOCKER	terraform:S8793	infra/terraform/modules/rds/main.tf:66	66
MAJOR	terraform:S6303	infra/terraform/modules/rds/main.tf:67	67
MAJOR	terraform:S6249	infra/terraform/modules/s3/main.tf:7	7
MAJOR	terraform:S6258	infra/terraform/modules/s3/main.tf:7	7
CRITICAL	terraform:S6281	infra/terraform/modules/s3/main.tf:28	28
MINOR	terraform:S6249	infra/terraform/modules/s3/main.tf:29	29
MAJOR	terraform:S6258	infra/terraform/modules/s3/main.tf:29	29
CRITICAL	terraform:S6281	infra/terraform/modules/s3/main.tf:34	34
MINOR	terraform:S6251	infra/terraform/modules/app/main.tf:68	68
MINOR	docker:S6471	services/auth-service/Dockerfile:2	2
MAJOR	docker:S8641	services/auth-service/Dockerfile:7	7
MAJOR	docker:S8544	services/auth-service/Dockerfile:7	7
CRITICAL	python:S4502	services/auth-service/app.py:12	12
CRITICAL	python:S4790	services/auth-service/app.py:32	32
MAJOR	python:S6946	services/auth-service/app.py:38	38
BLOCKER	python:S8392	services/auth-service/app.py:162	162
MINOR	python:S4507	services/auth-service/app.py:162	162
MINOR	docker:S6471	services/frontend/Dockerfile:2	2
MAJOR	docker:S8641	services/frontend/Dockerfile:7	7
MAJOR	docker:S8544	services/frontend/Dockerfile:7	7
CRITICAL	python:S4502	services/frontend/app.py:18	18
MINOR	python:S5332	services/frontend/app.py:16	16
MINOR	python:S5332	services/frontend/app.py:17	17
MAJOR	python:S6965	services/frontend/app.py:75	75
MAJOR	python:S6965	services/frontend/app.py:119	119
MAJOR	python:S6965	services/frontend/app.py:173	173
MAJOR	python:S6965	services/frontend/app.py:179	179
BLOCKER	python:S8392	services/frontend/app.py:185	185
MINOR	python:S4507	services/frontend/app.py:185	185
MINOR	docker:S6471	services/transaction-service/Dockerfile:2	2
MAJOR	docker:S8641	services/transaction-service/Dockerfile:7	7
MAJOR	docker:S8544	services/transaction-service/Dockerfile:7	7
CRITICAL	python:S4502	services/transaction-service/app.py:10	10
MINOR	python:S5332	services/transaction-service/app.py:12	12
MAJOR	python:S6946	services/transaction-service/app.py:67	67
BLOCKER	python:S8392	services/transaction-service/app.py:201	201
MINOR	python:S4507	services/transaction-service/app.py:201	201

```

onejuwakotie@MacBookAir SecureFlow-VulnerableCodebase %
```

Figure 4: SonarQube findings surfaced through the pipeline, including CRITICAL/BLOCKER severity application-code issues.

4.3 Container and Infrastructure Scanning

Trivy was adopted as a single tool covering both container image CVEs and Kubernetes configuration misconfiguration, reducing the number of distinct scanners the pipeline depends on.

Report Summary			
Target	Type	Vulnerabilities	Secrets
python:3.9-slim (debian 13.1)	debian	300	-
usr/local/lib/python3.9/site-packages/pip-23.0.1.dist-info/METADATA	python-pkg	6	-
usr/local/lib/python3.9/site-packages/setuptools-79.0.1.dist-info/METADATA	python-pkg	1	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/autocommand-2.2.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/backports.tarfile-1.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/importlib_metadata-8.0.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/inifect-7.3.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/jaraco.collections-5.1.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/jaraco.context-5.3.0.dist-info/METADATA	python-pkg	1	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/jaraco.funtools-4.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/jaraco.text-3.12.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/more_itertools-10.3.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/packaging-24.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/platformdirs-4.2.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/tomli-2.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/typeguard-4.3.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/typing_extensions-4.12.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/wheel-0.45.1.dist-info/METADATA	python-pkg	1	-
usr/local/lib/python3.9/site-packages/setuptools/_vendor/zip-3.19.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/wheel-0.45.1.dist-info/METADATA	python-pkg	1	-
Legend: - -: Not scanned - 0: Clean (no security findings detected)			
python:3.9-slim (debian 13.1)			
Total: 300 (UNKNOWN: 28, LOW: 126, MEDIUM: 98, HIGH: 41, CRITICAL: 7)			

Figure 5: Local Trivy scan summary against the initial, unremediated container images, the baseline used for all subsequent before/after comparisons.

4.4 The Differentiated Security Gate

A real problem surfaced during Week 1: a uniform “fail on any CRITICAL finding” rule either blocks delivery over application-layer findings an infrastructure engineer cannot fix, or fails to block delivery over infrastructure findings that should never ship. The Security Gate resolves this by aggregating every scanner's artifact and applying an ownership-based policy, documented in full in Section 7.

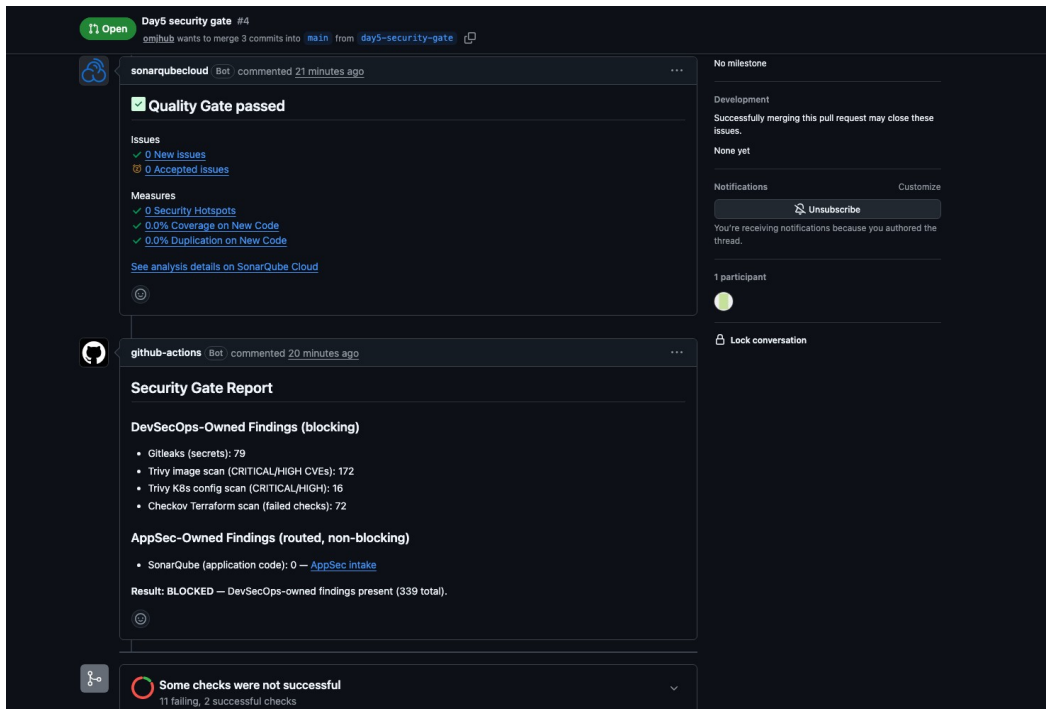


Figure 6: The Security Gate's differentiated PR comment, separating DevSecOps-owned (blocking) findings from AppSec-owned (routed, non-blocking) findings.

5. Week 2 Implementation: Remediation, Enforcement, Proof

Week 2 shifted from detection to making fixes real, un-bypassable, and continuously provable, secrets management, admission-time policy enforcement, live runtime detection, and full remediation of every detected category.

5.1 Secrets Management with Vault

HashiCorp Vault was deployed with Kubernetes authentication, replacing plaintext environment variables and a shared ConfigMap with per-service, read-only, audited secret access.

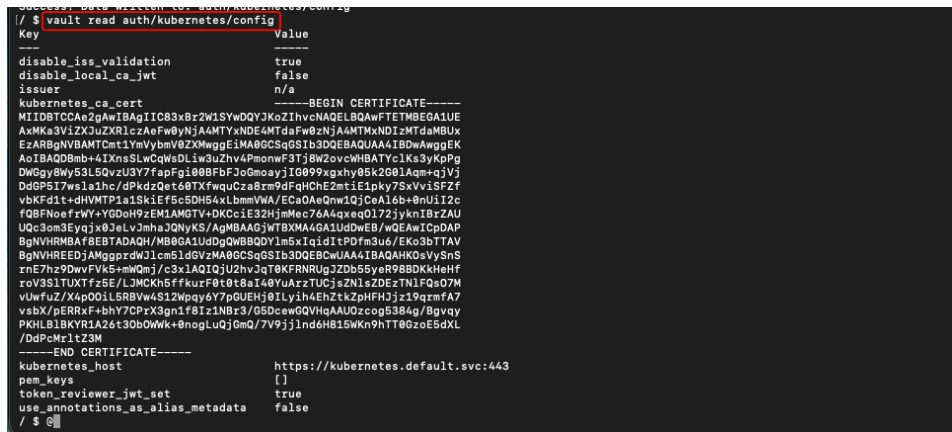


Figure 7: Vault's Kubernetes authentication method, configured and verified, the foundation for every subsequent per-service secret policy.

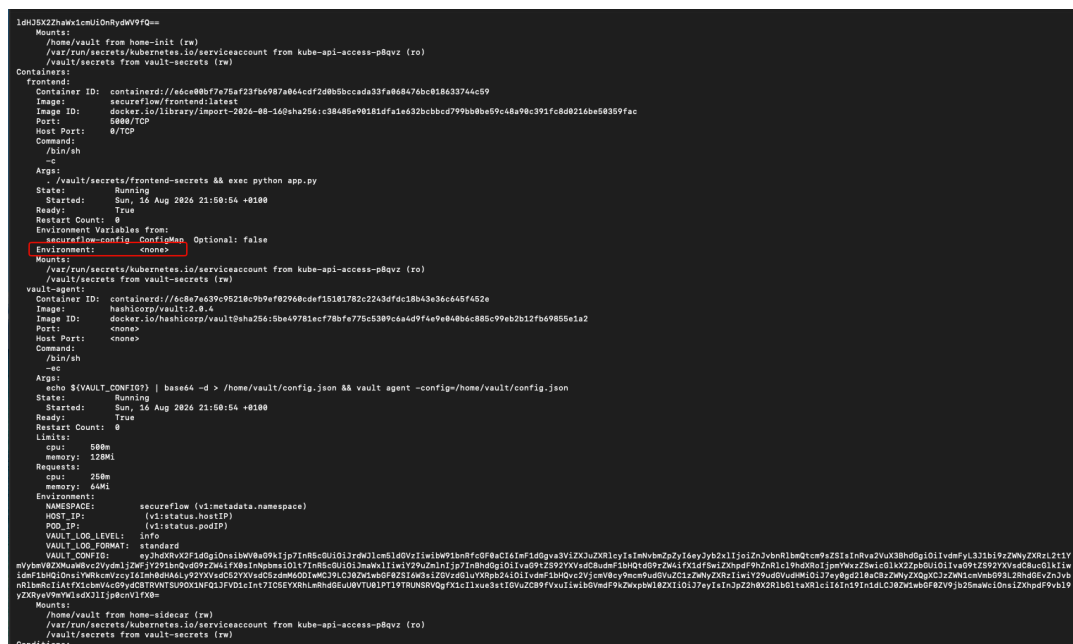


Figure 8: `kubectl describe pod` confirming zero plaintext secrets in the declared pod specification, credentials are delivered exclusively via the Vault Agent Injector.

5.2 Git History Remediation

A full git-filter-repo history rewrite was performed to remove real committed secrets from every historical commit, independently re-verified clean on three separate occasions after an initial false alarm traced to a stale local artifact was correctly diagnosed rather than accepted at face value.

```

omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % git remote -v
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % git log --all --oneline -- .env .env.txt
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % git log --all --oneline -- docker-compose.yml | grep -E "authpass123|txpass123|changeme|super-secret-key-123"
43
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 9: Post-rewrite verification confirming zero real secret values remain in any commit reachable in current history.

5.3 Admission-Time Policy Enforcement, OPA Gatekeeper

OPA Gatekeeper was installed to reject non-compliant Kubernetes resources at admission time, before they are ever created, a fundamentally different enforcement point from every scanner used so far, which report after the fact.

```

omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % sleep 30
[kubectrl get pods -n gatekeeper-system
NAME                                READY   STATUS    RESTARTS   AGE
gatekeeper-audit-b9d5f797b-jwgib   1/1     Running   0           3m32s
gatekeeper-controller-manager-5b6f9b6b9-4jrfw  1/1     Running   0           3m32s
gatekeeper-controller-manager-5b6f9b6b9-k29ss  1/1     Running   0           3m32s
gatekeeper-controller-manager-5b6f9b6b9-qzhcw  1/1     Running   0           3m32s
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 10: OPA Gatekeeper controller and audit components running, ready to evaluate every resource against the project's four ConstraintTemplates.

5.4 Network Segmentation

Default-deny NetworkPolicies with explicit service-to-service allow-lists were authored for all five workloads, verified with live traffic in both directions, confirming the allowed path succeeds and an unauthorised path is genuinely blocked, not merely configured.

```

omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % curl -i http://localhost:5000/login \
-X POST \
-H "Host: localhost:5000" \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "Content-Length: 287" \
-d "username=admin&password=admin123" \
Content-Type: text/html; charset=utf-8
Content-Length: 287
Location: /dashboard
Vary: Cookie
Set-Cookie: session=e3XVjsdGjAURf-layVCyAg78YcoXh8NPhWwgo8K1wVD0-uys3z2Zz8JUF8vUa5i3i5lnePh3JIG2y8BHfVqWdyKw-hbrqH8Kuo1w2i5sc8em-tg0n9xUhwXKJgpbCY9u5H_n3god2cbrqP8-U0ryQJ2H2z8d4WJRQ0Qd182j;uk71IRC19T19Hf1CTLYVYy
p_3TODW0d0Pva-cA.aakWw.RanAp1221k21P0ub-HyppGdu; httpOnly; Path: /
Connection: close

<doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: ca href="/dashboard"/>dashboard/</p>
<p>If not, click the link.</p>
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % kubectrl exec -it $kubectrl get pod -n secureflow -l svc=frontend -- jsonpath='{.items[0].metadata.name}' | xargs -n 1 secureflow -- frontend -- python3 -c "
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(3)
try:
    s.connect(('auth-db', 5432))
    print('CONNECTED - network policy NOT blocking this (should be blocked)')
except Exception as e:
    print('BLOCKED as expected:', e)
BLOCKED as expected: timed out
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 11: Live verification of NetworkPolicy enforcement, the allowed frontend→auth-service→auth-db login flow succeeding, and a direct frontend→auth-db connection attempt timing out as intended.

5.5 Runtime Detection, Falco

Falco was deployed with eBPF-based syscall monitoring and four custom rules authored for SecureFlow's specific threat model, layered on top of Falco's own extensive default rule set.

```

omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % sleep 20
[kubectrl get pods -n falco
NAME                                READY   STATUS    RESTARTS   AGE
falco-vtbd1 2/2 Running 0 117s
Defaulted container "falco" out of: falco, falcoctl-artifact-follow, falcoctl-artifact-install (init)
Wed Aug 19 18:31:35 2026: Loading rules from:
Wed Aug 19 18:31:35 2026: /etc/falco/falco_rules.yaml | schema validation: ok
Wed Aug 19 18:31:35 2026: /etc/falco/rules.d/secureflow_rules.yaml | schema validation: ok
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 12: Falco confirming both the default rule set and SecureFlow's custom rule file loaded and schema-validated.

5.6 Full Database Vault Migration and Hardening

In Week 2's later stages, both PostgreSQL databases were migrated onto Vault-delivered credentials (closing the last remaining plaintext-ConfigMap gap in the project) and hardened with a read-only root filesystem, resolved through a genuinely difficult Kubernetes init-container ownership and capability puzzle before reaching a stable, secure, and functional configuration.

```

omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % kubectl apply -k infra/kubernetes/base/
kubectl delete pod -n secureflow -l svc=auth-db
kubectl delete pod -n secureflow -l svc=transaction-db
sleep 30
[kubectl get pods -n secureflow
namespace/secureflow unchanged
configmap/secureflow-config unchanged
service/auth-db unchanged
service/auth-service unchanged
service/frontend unchanged
service/transaction-db unchanged
service/transaction-service unchanged
deployment.apps/auth-db configured
deployment.apps/auth-service unchanged
deployment.apps/frontend unchanged
deployment.apps/transaction-db configured
deployment.apps/transaction-service unchanged
networkpolicy.networking.k8s.io/auth-db-ingress unchanged
networkpolicy.networking.k8s.io/auth-service-egress configured
networkpolicy.networking.k8s.io/auth-service-ingress unchanged
networkpolicy.networking.k8s.io/default-deny-all unchanged
networkpolicy.networking.k8s.io/frontend-egress configured
networkpolicy.networking.k8s.io/transaction-db-ingress unchanged
networkpolicy.networking.k8s.io/transaction-service-egress configured
networkpolicy.networking.k8s.io/transaction-service-ingress unchanged
pod "auth-db-6bfcd9f4bdf-tqm7l" deleted from secureflow namespace
pod "auth-db-b977996bc-vd7kz" deleted from secureflow namespace
pod "transaction-db-5449f94bdf-tqm7l" deleted from secureflow namespace
pod "transaction-db-6b656d7fbb-44zxt" deleted from secureflow namespace
NAME                                READY    STATUS    RESTARTS   AGE
auth-db-6bfcd9f4bdf-p5lts           2/2     Running   0           46s
auth-service-679c89676b-nmqcv       2/2     Running   0           54m
frontend-979bf67b-g4zqk             2/2     Running   0           54m
transaction-db-6b656d7fbb-slnb9     2/2     Running   0           62s
transaction-service-7566ff54f9-6dftj 2/2     Running   0           53m
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %

```

Figure 13: Both PostgreSQL databases running with Vault-delivered credentials and full securityContext hardening, confirmed healthy.

6. Security Assessment: Before / After

The table below states before/after status per finding category, with remediation or routing status noted precisely. Full per-finding detail is preserved in the JSON tool reports archived alongside this report.

Category	Before	After	Status / Routing
Secrets (Gitleaks)	Multiple real secrets in git history	0 blocking findings	Remediated, git-filter-repo rewrite, verified 3x independently
Application code (SonarQube)	55 findings incl. 12 CRITICAL/BLOCKER	26–33 findings (fluctuates with codebase)	Routed, AppSec-owned, non-blocking by design
Container image CVEs (Trivy)	172 findings, 56–58 CRITICAL/HIGH per service	0 blocking (all remaining CVEs confirmed no fix available)	Remediated, Flask/Werkzeug major upgrade, OS package upgrade
Kubernetes config (Trivy K8s)	16 findings incl. privileged containers, mutable tags	0 blocking findings	Remediated, securityContext hardening incl. Vault-injected containers
Terraform IaC (Checkov)	124 findings (72 failed checks)	0 failed checks	Remediated, IAM, EKS, S3, VPC, RDS; 6 items via documented skip-list
Runtime / DAST (ZAP)	Not previously tested live	151 findings; 2 confirmed HIGH (SQLi), 2 low-confidence unconfirmed	Remediated finding routed to AppSec; SQLi matches Week 1 manual exploit exactly
Secrets delivery (Vault)	Plaintext env vars / ConfigMap	Vault Agent Injector on all 5 workloads incl. databases	Remediated, verified via audit log and live pod inspection
Network segmentation	Fully open pod-to-pod traffic	Default-deny + explicit allow-list, all 5 workloads	Remediated, verified with live allowed and blocked traffic tests

7. CI/CD Pipeline and the Security Gate

7.1 Final Verified Pipeline Result

Run reference: github.com/omjhub/SecureFlow-VulnerableCodebase/actions/runs/34749258295

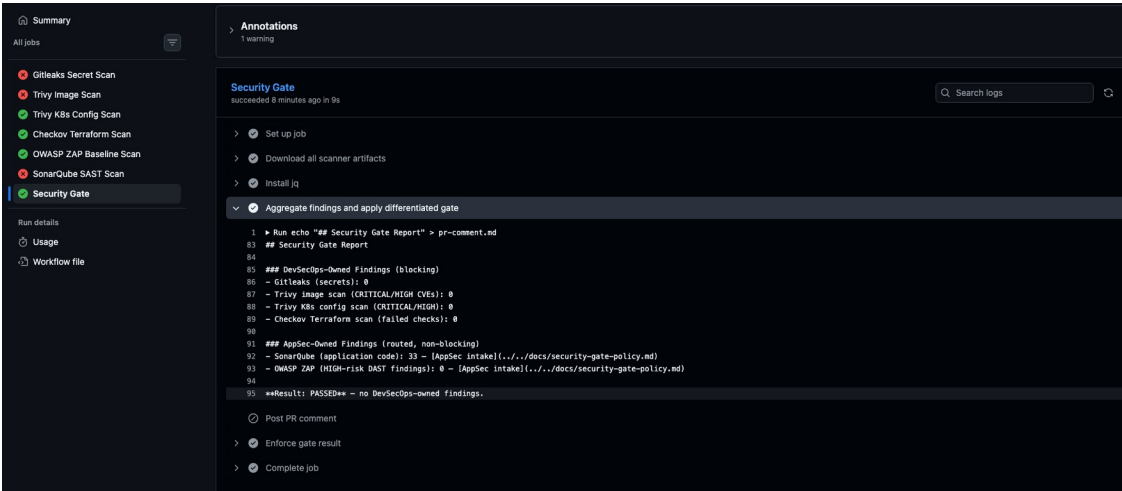


Figure 14: Final Security Gate run, Result: PASSED, no DevSecOps-owned findings, with OWASP ZAP now fully integrated as a real pipeline stage.

7.2 The Ownership Matrix

Scanner	Domain	Owner	Category
Gitleaks	Secrets in git history	DevSecOps	Any committed credential
Trivy (image / K8s)	Container CVEs / K8s misconfig	DevSecOps	CK-*
Checkov	Terraform / cloud infrastructure	DevSecOps	IV-*
SonarQube	Application source (SAST)	AppSec	AV-*, TV-*, FV-*
OWASP ZAP	Running application (DAST)	AppSec	AV-*, TV-*, FV-*

7.3 Documented Gate Exceptions

- Gitleaks findings matching the literal string “REDACTED” are excluded, confirmed-safe placeholder matches from the history rewrite, independently verified three times to contain zero real secret values.
- Trivy CVEs with no FixedVersion available upstream are excluded from the blocking count, since a gate that can never be satisfied provides no remediation signal. Any CVE with a real, available fix remains fully blocking.

8. Runtime Security: Falco Evidence

Four custom rules were authored and each trigger scenario deliberately reproduced against the live cluster to confirm detection, not merely assumed from the rule's syntax.

Trigger Scenario	Result	Evidence
Shell spawned in container	Detected	Confirmed via Falco's built-in equivalent rule; custom rule's independent non-firing documented, not concealed
Sensitive file read (/etc/shadow)	Confirmed, custom rule fired	The read itself was separately blocked by Day 7 hardening, two independent layers both functioning on the same event
Unexpected outbound connection	Confirmed, exact match	Captured at the syscall layer independently of the NetworkPolicy that separately blocked the connection
Package manager execution	Confirmed, multiple matches	Captured across every retry as the network layer independently blocked the install

Falco's native Prometheus metrics were subsequently enabled and are visualised live in the Grafana dashboard (Section 11), closing the loop from “rule fires once” to “continuously observable over time.”

9. Secrets Management: Vault

9.1 Verified Chain

- Kubernetes ServiceAccount authenticates to Vault via the Kubernetes auth method.
- Per-service policy grants read-only, own-path-only access, confirmed via direct policy inspection.
- Vault Agent Injector delivers secrets as files, consumed by each service at startup, never as a plaintext environment variable.
- Vault's audit log HMAC-hashes secret values, confirmed directly, even Vault's own logs never contain plaintext secrets.
- Extended to both PostgreSQL databases in Week 2, closing the last remaining plaintext-credential gap in the project.

9.2 Honestly Documented Gaps

- Vault runs in development mode (in-memory storage). Appropriate for this local engagement, but all configuration is lost on any Vault pod restart, encountered directly during two separate incidents and documented as a known, real characteristic, not treated as a surprise each time it recurred.
- Vault roles do not have an explicit JWT audience configured. Vault itself warns of this on every role write; not required for function, flagged as a hardening opportunity.

10. Dynamic Application Security Testing

OWASP ZAP was run in two configurations: an unauthenticated full scan, and a properly authenticated full scan using ZAP's forced-user mode, configured via the ZAP API. Since no legitimate password satisfies the application's broken password hashes, ZAP was authenticated using the SQL injection payload itself (admin'--) as the login username, the only genuinely working credential path, letting the authenticated crawl reach pages the unauthenticated scan never discovered.

10.1 Confirmed Findings

- SQL Injection + SQL Injection – Authentication Bypass (/login), CONFIRMED, high confidence. Independently rediscovered by ZAP's automated scanner, exactly matching the manual exploit proven in Week 1.
- Path Traversal x2 (/transfer), NOT CONFIRMED. Both instances report ZAP's own confidence as Low with an empty evidence field. Documented for completeness but not counted as a confirmed vulnerability, correctly distinguishing scanner signal from scanner noise rather than accepting every HIGH-labelled result at face value.

```
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % python3 -c "
import json
with open('/Users/omajuwaokotie/zap-scan/zap-full-authenticated-alerts.json') as f:
    data = json.load(f)
    alerts = data['alerts']
from collections import Counter
by_risk = Counter(a['risk'] for a in alerts)
for risk, count in by_risk.most_common():
    print(f'{risk}: {count}')
"
```

Informational: 67
Low: 41
Medium: 39
High: 4

Figure 15: ZAP authenticated scan finding summary, 151 total alerts, 4 raw HIGH, of which 2 are independently confirmed real and 2 correctly identified as low-confidence.

10.2 Critical Discovery, Live Werkzeug Interactive Debugger

During spider crawl analysis, ZAP surfaced file-path-like URLs sourced from Werkzeug debugger stack traces. Direct verification confirmed HTTP 200 with the genuine, live Werkzeug interactive debugger JavaScript, not a spider artifact. This implements a remote Python code-execution console, gated only by a PIN of unverified strength.

This finding elevates a previously-documented information-disclosure issue (stack traces revealing file paths) to a CRITICAL, RCE-adjacent finding. The PIN itself was deliberately not tested or bypassed, the finding did not require active exploitation beyond what a passive scan plus direct verification already proved.

Remediation (AppSec-owned): set debug=False / FLASK_ENV=production for any non-local deployment, a one-line application-code change, routed via the standard AppSec intake process per the project's ownership matrix.

11. Observability: Grafana Dashboard

A single “SecureFlow Security Overview” dashboard was built with six independently verified, live panels, each backed by a PromQL query checked directly against Prometheus before being wired into a visualisation.

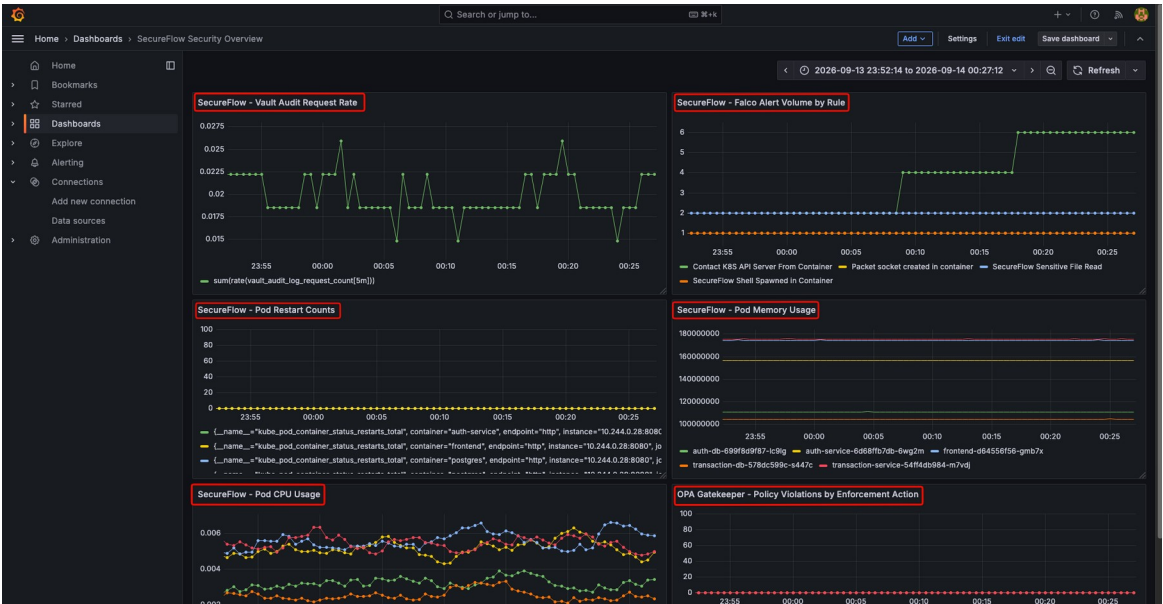


Figure 16: The completed SecureFlow Security Overview dashboard, Vault audit rate, Falco alert volume by rule, pod restart counts, pod memory usage, pod CPU usage, and OPA Gatekeeper policy violations, all live.

Panel	Query / Source
OPA Gatekeeper, policy violations	gatekeeper_violations
Pod CPU usage	sum(rate(container_cpu_usage_seconds_total{namespace="secureflow"}[5m])) by (pod)
Pod memory usage	sum(container_memory_working_set_bytes{namespace="secureflow"}) by (pod)
Pod restart counts	kube_pod_container_status_restarts_total{namespace="secureflow"}
Falco alert volume by rule	sum(falcosecurity_falco_rules_matches_total) by (rule_name)
Vault audit request rate	sum(rate(vault_audit_log_request_count[5m]))

Two additional panels named in the original scope (CI/CD pipeline trend, AppSec finding volume over time) require data from GitHub Actions and SonarQube, neither natively scraped by Prometheus. Documented in Section 14 as a scoped follow-up rather than forced through a version upgrade that risked the six already-working panels.

12. Results Against Success Metrics

Metric	Baseline	Final	Change
Checkov failed checks	72	0	-100%
Trivy image CRITICAL/HIGH (blocking)	56-58 per service	0	-100% blocking (raw CVEs --52% before final pass)
Trivy K8s CRITICAL/HIGH	16	0	-100%
Gitleaks blocking findings	80+ (pre-scrub)	0	-100%, verified 3x independently
Security Gate result	BLOCKED	PASSED	Full pass, all DevSecOps- owned categories
Live Grafana panels	0	6	New capability
Falco custom rules confirmed firing	0	3 of 4 (4th via built-in equivalent)	New capability
DAST HIGH findings confirmed real	Untested	2 (SQLi, matching known exploit)	New verification of a known issue

13. Challenges and Lessons Learned

13.1 A Passing Result Is Not Proof

Multiple times, a clean-looking result concealed a real problem: a null severity field silently defeating an early Checkov filter; a stale local artifact that briefly looked like a re-introduced secret; a Grafana dashboard that looked complete until the pod restarted. Every one was only caught by checking raw data directly rather than trusting a summary.

13.2 Multiple Layers Catch What One Misses

Trivy missed a mutable image tag issue that Gatekeeper's admission-time constraint caught live. A NetworkPolicy blocked a connection that Falco still correctly logged at the syscall level, proof both layers were independently functioning on the same event, not redundant, but mutually reinforcing.

13.3 Ephemeral-by-Default Components Need Explicit Persistence

Both Vault (dev-mode in-memory storage) and Grafana (no default PersistentVolumeClaim) silently lost real, hard-won configuration on restart, discovered independently in two different components on two different days. Neither was a bug, both were correct, documented default behaviours that were not accounted for until a restart actually happened.

13.4 A Full, Unexplained Cluster Loss, and Recovery from Code Alone

Mid-engagement, every Docker container across every local project vanished with no clear trigger identified. Recovery was achieved entirely from committed Kubernetes manifests and documented procedure, no manual reconstruction, no guesswork. The full sequence was consolidated into a single reusable rebuild script immediately afterward, so a repeat incident requires one command rather than a re-derived manual sequence.

13.5 CVE Data Is Living, Not Static

A CVE previously confirmed to have no available fix (perl-base) genuinely acquired a real Debian patch mid-engagement. Dependency data, and even a Kubernetes admission webhook's certificate trust relationship, are not fixed facts, they change, sometimes silently, and a remediation project needs to periodically re-verify rather than assume a prior finding stays true forever.

13.6 Honest Gaps Are Stronger Evidence Than False Completeness

Throughout this engagement, findings that could not be confirmed, gaps that could not be closed without disproportionate risk, and deliberate design tradeoffs are named precisely rather than hidden. Every claim in this report can be verified against the referenced raw tool output.

14. Production Gaps and Future Enhancements

- Cosign image signing and SBOM generation, not yet implemented; digest pinning is functional via a local registry but not cryptographically signed.
- Grafana panels for CI/CD trend and AppSec finding volume over time require a Grafana version upgrade or a Prometheus Pushgateway integration against the GitHub Actions and SonarQube APIs.
- RDS's own master credential remains sourced from a Terraform variable rather than Vault; application-tier secrets are fully migrated, the database-provisioning credential is a documented, scoped follow-up.
- The live Werkzeug interactive debugger (Section 10.2) requires an AppSec-owned application-code fix, the single highest-priority open item in this report.
- One Falco custom rule was not independently confirmed firing on its own, despite the underlying event being confirmed detected via Falco's built-in equivalent rule. Root cause not conclusively identified; documented rather than silently resolved.
- Root cause of the mid-engagement full-cluster and Docker state loss was never conclusively identified. Recovery was complete and verified; the underlying trigger remains unknown.

15. Conclusion

This engagement set out to prove that a vulnerable system can be systematically hardened through an automated, ownership-aware pipeline rather than through ad hoc manual fixes, and to apply the judgement to know the difference between a genuinely resolved finding and one that only looks resolved. Every headline number in this report traces back to a real tool run, a real screenshot, or a real, reproducible command. Where something could not be fixed, could not be confirmed, or was deliberately deferred, that is stated plainly rather than smoothed over. That discipline, verify before trusting, document the gap rather than hide it, is the actual deliverable of this engagement, more than any single passing pipeline run.

16. Appendix: Supplementary Evidence

This appendix archives additional evidence not embedded in the main body of the report, organised by phase. Every image is a direct, unedited screenshot captured during the engagement.

16.1 Initial Assessment and Manual Exploitation

```

❯ docker-compose --dry-run --build --no-build --no-build-remote
[+] Building 0.0s
=> [auth-service] exporting to image
=> exporting layers
=> writing image sha256:18f91fa5c873f6dbab6332f384c850764cfcd0b7c3d6ad572a6abfcfc
=> naming to docker.io/library/secureflow-vulnerablelabcodebase-auth-service
=> [transaction-service] resolving provenance for metadata file
=> [auth-service] resolving provenance for metadata file
[+] 0/0
Image secureflow-vulnerablelabcodebase-frontend      Null 0.0s
Image secureflow-vulnerablelabcodebase-auth-service   Null 0.0s
Image secureflow-vulnerablelabcodebase-transaction-service Null 0.0s
Network secureflow-vulnerablelabcodebase_default      Created 0.0s
Container secureflow-vulnerablelabcodebase-auth-db-1   Created 0.0s
Container secureflow-vulnerablelabcodebase-transaction-db-1 Created 0.0s
Container secureflow-vulnerablelabcodebase-auth-service-1 Created 0.0s
Container secureflow-vulnerablelabcodebase-transaction-service-1 Created 0.0s
Container secureflow-vulnerablelabcodebase-frontend-1 Created 0.0s
Attaching to auth-db-1, auth-service-1, frontend-1, transaction-service-1
auth-db-1
auth-db-1
auth-db-1
PostgreSQL Database directory appears to contain a database; Skipping initialization
transaction-db-1
transaction-db-1
transaction-db-1
PostgreSQL Database directory appears to contain a database; Skipping initialization
transaction-db-1
2024-07-26 18:16:15.106 UTC [1] [log]: starting PostgreSQL 14.23 (Debian 14.23-0.pgdg31n1) on arch46-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
auth-db-1
2024-07-26 18:16:15.106 UTC [1] [log]: listening on IPv4 address "0.0.0.0", port 5432
auth-db-1
2024-07-26 18:16:15.106 UTC [1] [log]: listening on IPv6 address "*", port 5432
auth-db-1
2024-07-26 18:16:15.106 UTC [1] [log]: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
transaction-db-1
2024-07-26 18:16:15.109 UTC [26] [log]: database system was shut down at 2024-07-26 18:15:48 UTC
auth-db-1
2024-07-26 18:16:15.110 UTC [1] [log]: starting PostgreSQL 14.23 (Debian 14.23-0.pgdg31n1) on arch46-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
transaction-db-1
2024-07-26 18:16:15.110 UTC [1] [log]: listening on IPv4 address "0.0.0.0", port 5432
transaction-db-1
2024-07-26 18:16:15.110 UTC [1] [log]: listening on IPv6 address "*", port 5432
transaction-db-1
2024-07-26 18:16:15.113 UTC [1] [log]: database system is ready to accept connections
transaction-db-1
2024-07-26 18:16:15.113 UTC [1] [log]: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
transaction-db-1
2024-07-26 18:16:15.114 UTC [26] [log]: database system was shut down at 2024-07-26 18:15:48 UTC
auth-service-1
2024-07-26 18:16:15.114 UTC [1] [log]: database system is ready to accept connections
auth-service-1
* Serving Flask app "app"
auth-service-1
* Debug mode: on
auth-service-1
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
auth-service-1
* Running on all addresses (0.0.0.0)
auth-service-1
* Running on http://127.0.0.1:5000
auth-service-1
* Running on http://172.18.0.4:5000
auth-service-1
Press CTRL+C to quit
auth-service-1
* Restarting with stat
transaction-service-1
* Serving Flask app "app"
transaction-service-1
* Debug mode: on
transaction-service-1
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
transaction-service-1
* Running on all addresses (0.0.0.0)
transaction-service-1
* Running on http://127.0.0.1:5000
transaction-service-1
* Running on http://172.18.0.5:5000
transaction-service-1
Press CTRL+C to quit
transaction-service-1
* Restarting with stat
auth-service-1
* Debugger is active
auth-service-1
* Debugger PIN: 132-888-413
transaction-service-1
* Serving Flask app "app"
transaction-service-1
* Debug mode: on
transaction-service-1
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
transaction-service-1
* Running on all addresses (0.0.0.0)
transaction-service-1
* Running on http://127.0.0.1:5000
transaction-service-1
* Running on http://172.18.0.6:5000
transaction-service-1
Press CTRL+C to quit
transaction-service-1
* Restarting with stat
transaction-service-1
* Debugger is active
transaction-service-1
* Debugger PIN: 132-173-549
transaction-service-1
* Debugger is active
transaction-service-1
* Debugger PIN: 348-822-492
transaction-service-1
192.168.65.1 - [28/Jul/2024 18:50:33] "GET / HTTP/1.1" 302 -
transaction-service-1
192.168.65.1 - [28/Jul/2024 18:50:33] "GET /login HTTP/1.1" 200 -
transaction-service-1
192.168.65.1 - [28/Jul/2024 18:50:33] "GET /api/users HTTP/1.1" 404 -

```

Figure A1: All five services (three application, two database) running at engagement start.

```

omejuwakotic@MacBookAir: SecureFlow-VulnerableCodebase $ curl -i -X POST http://localhost:6881/login \
-H "Content-Type: application/json" \
-d '{"username": "alice"',"-","password":"anything"}'
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Thu, 30 Jul 2024 19:08:38 GMT
Content-Type: application/json
Content-Length: 196
Connection: close

{"role": "user",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IHYXZjkiLCJhbGciOiJIUzI1NiIsbnNpdCI6ImFsaWNoIiwiaWF0Ij0iMTY2OTM1f6LcfmBg3glA.3yeQJ7VBSzuV-jjfUsCHQ".
"user_id": 2 }

omejuwakotic@MacBookAir: SecureFlow-VulnerableCodebase $ ALICE_TOKEN='eyJhbGciOiJIUzI1NiIsInR5cCI6IHYXZjkiLCJhbGciOiJIUzI1NiIsbnNpdCI6ImFsaWNoIiwiaWF0Ij0iMTY2OTM1f6LcfmBg3glA.3yeQJ7VBSzuV-jjfUsCHQ'
curl -i http://localhost:6882/balance/ \
-H "Authorization: Bearer $ALICE_TOKEN"
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Thu, 30 Jul 2024 19:01:52 GMT
Content-Type: application/json
Content-Length: 60
Connection: close

{"account_id": 3,
"balance": 1200.0,
"owner_id": 3 }

omejuwakotic@MacBookAir: SecureFlow-VulnerableCodebase $ ALICE_TOKEN='eyJhbGciOiJIUzI1NiIsInR5cCI6IHYXZjkiLCJhbGciOiJIUzI1NiIsbnNpdCI6ImFsaWNoIiwiaWF0Ij0iMTY2OTM1f6LcfmBg3glA.3yeQJ7VBSzuV-jjfUsCHQ'
curl -i http://localhost:6882/balance/2 \
-H "Authorization: Bearer $ALICE_TOKEN"
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Thu, 30 Jul 2024 19:02:39 GMT
Content-Type: application/json
Content-Length: 60
Connection: close

{"account_id": 2,
"balance": 500.0,
"owner_id": 2 }

omejuwakotic@MacBookAir: SecureFlow-VulnerableCodebase $

```

Figure A2: Manual proof of an insecure direct object reference, allowing access to another account's data.

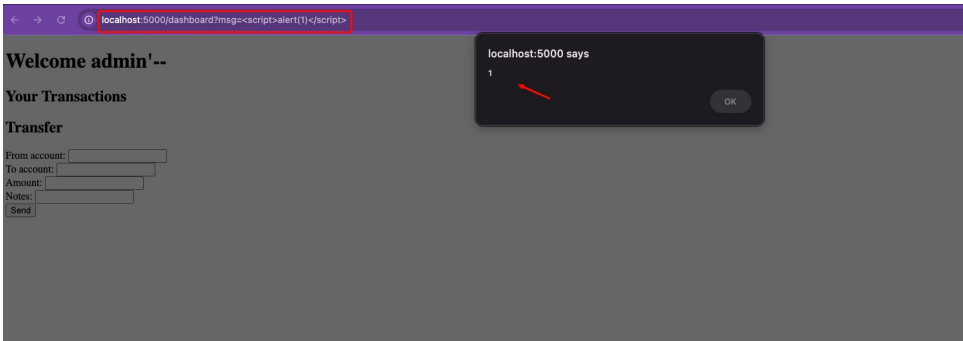


Figure A3: Manual proof of a reflected cross-site scripting vulnerability.

16.2 CI/CD Pipeline and the Security Gate

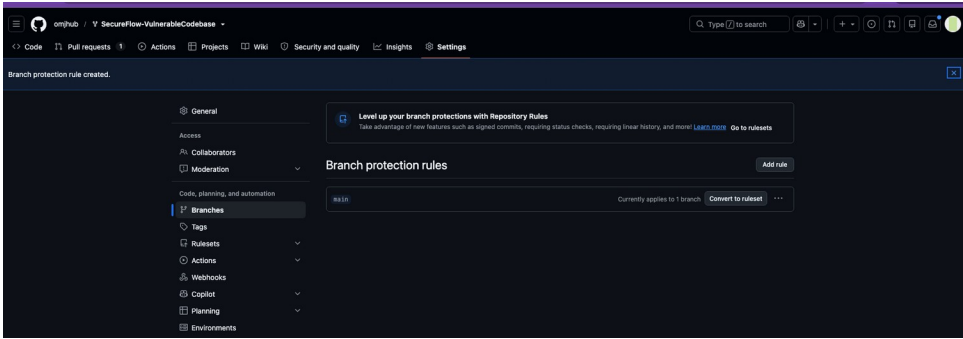


Figure A4: Branch protection configuration requiring the Security Gate to pass before a merge is permitted.

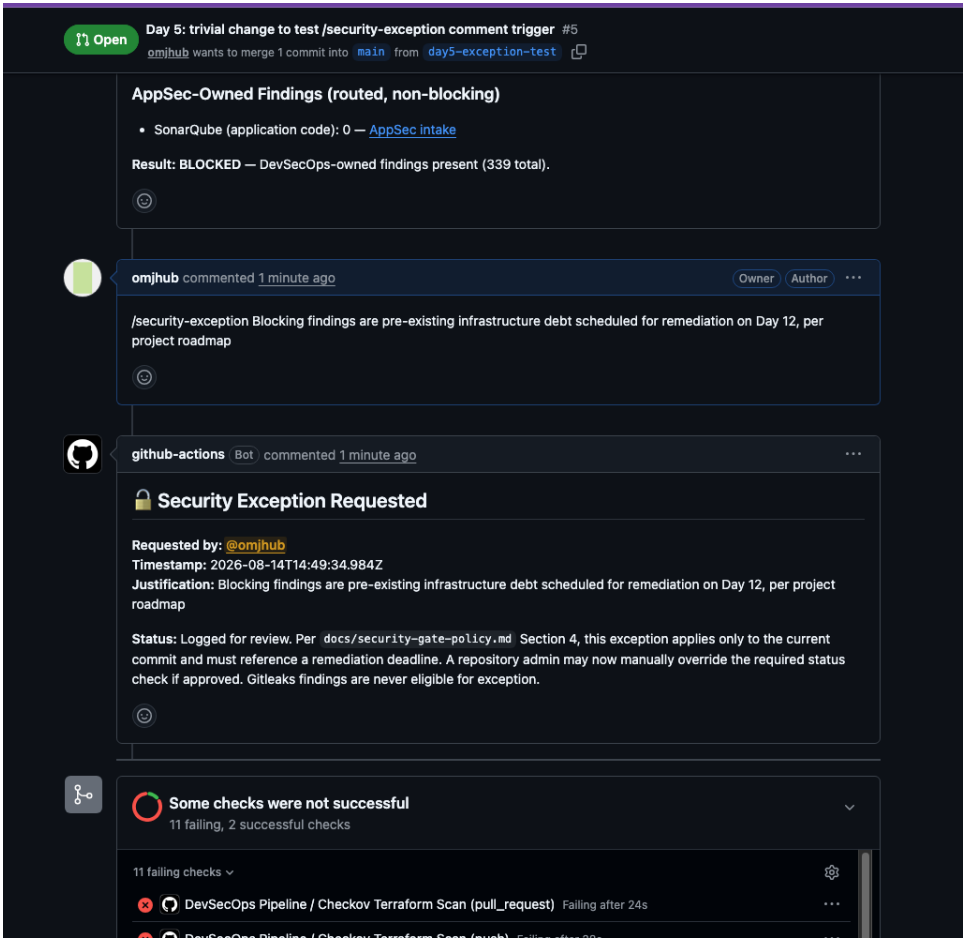


Figure A5: A logged, justified security exception, recorded with full audit trail per the documented exception process.

16.3 Secrets Management

```

root@kali:~# vault kv get secureflow/auth-service
===== Secret Path =====
secureflow/data/auth-service

===== Metadata =====
Key                           Value
-----
created_time                   2026-08-16T16:40:03.201760426Z
custom_metadata                <nil>
deletion_time                  n/a
destroyed                      false
version                        1

===== Data =====
Key                           Value
-----
DB_PASSWORD                    authpass123
JWT_SECRET                     super-secret-key-123
/ $ @

```

Figure A6: A secret written to Vault's key-value engine, scoped to a single service's own path.

[illegible]

Figure A7: Vault audit log excerpt confirming secret values are HMAC-hashed, never stored or logged in plaintext.

16.4 Container and Infrastructure Remediation

[illegible]

Figure A8: Remediated container compose configuration, credentials removed from plaintext and image references pinned.

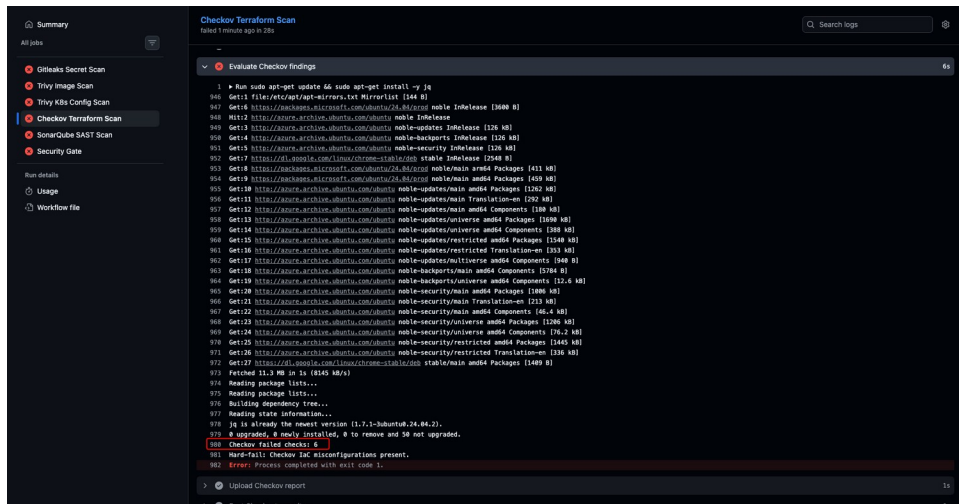


Figure A9: Final Checkov Terraform scan result following full remediation across IAM, EKS, S3, VPC, and RDS.

16.5 Runtime Security and Observability



Figure A10: Falco alert captured live during a deliberate package-manager-execution trigger test.

```
omajuwakotie@MacBookAir SecureFlow-VulnerableCodebase % kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0	2/2	Running	0	26m
prometheus-grafana-6d489fbc6f-qjkh2	3/3	Running	0	26m
prometheus-kube-prometheus-operator-5676f5bfd-rnxb2	1/1	Running	0	26m
prometheus-kube-state-metrics-bb554c984-5v18k	1/1	Running	0	26m
prometheus-prometheus-kube-prometheus-prometheus-0	2/2	Running	0	26m
prometheus-prometheus-node-exporter-81z65	1/1	Running	0	26m

```
omajuwakotie@MacBookAir SecureFlow-VulnerableCodebase %
```

Figure A11: Full monitoring stack (Prometheus, Grafana, Alertmanager) confirmed healthy after deployment.

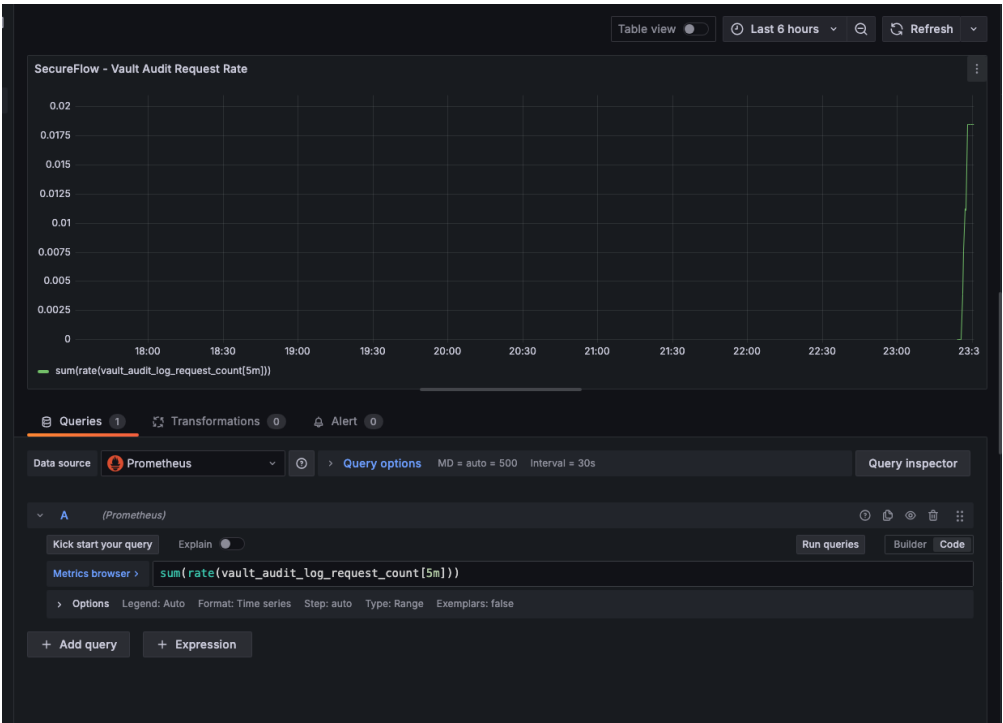


Figure A12: Grafana panel showing live Vault audit request rate during a runtime security test.

```
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase % kubectl apply -k infra/kubernetes/base/
kubectl delete pod -n secureflow -l svc=auth-service
kubectl delete pod -n secureflow -l svc=frontend
kubectl delete pod -n secureflow -l svc=transaction-service
kubectl delete pod -n secureflow -l svc=auth-db
kubectl delete pod -n secureflow -l svc=transaction-db
sleep 30
kubectl get pods -n secureflow
namespace/secureflow unchanged
configmap/secureflow-config unchanged
service/auth-db unchanged
service/auth-service unchanged
service/frontend unchanged
service/transaction-db unchanged
service/transaction-service unchanged
deployment.apps/auth-db configured
deployment.apps/auth-service configured
deployment.apps/frontend configured
deployment.apps/transaction-db configured
deployment.apps/transaction-service configured
networkpolicy.networking.k8s.io/auth-db-egress configured
networkpolicy.networking.k8s.io/auth-db-ingress unchanged
networkpolicy.networking.k8s.io/auth-service-egress configured
networkpolicy.networking.k8s.io/auth-service-ingress unchanged
networkpolicy.networking.k8s.io/default-deny-all unchanged
networkpolicy.networking.k8s.io/frontend-egress configured
networkpolicy.networking.k8s.io/transaction-db-egress configured
networkpolicy.networking.k8s.io/transaction-db-ingress unchanged
networkpolicy.networking.k8s.io/transaction-service-egress configured
networkpolicy.networking.k8s.io/transaction-service-ingress unchanged
pod "auth-service-5c5cb889bd-xz64b" deleted from secureflow namespace
pod "auth-service-644769cb59-67ldd" deleted from secureflow namespace
pod "frontend-67f99fc4-v98lp" deleted from secureflow namespace
pod "frontend-79cf6546bc-dlwhh" deleted from secureflow namespace
pod "transaction-service-84d587877-xhkd" deleted from secureflow namespace
pod "auth-db-7c5f747cf8-smpf9" deleted from secureflow namespace
pod "transaction-db-8d8769968-552cs" deleted from secureflow namespace
NAME                                READY    STATUS    RESTARTS    AGE
auth-db-7c5f747cf8-h68w7            2/2     Running  0           32s
auth-service-5c5cb889bd-jl9fm       2/2     Running  0           37s
frontend-67f99fc4-wlp8p             2/2     Running  0           36s
transaction-db-8d8769968-k2ntd      2/2     Running  0           31s
transaction-service-84d587877-96p2q 2/2     Running  0           33s
omajuwaokotie@MacBookAir SecureFlow-VulnerableCodebase %
```

Figure A13: Vault Agent sidecar container confirmed running with full securityContext hardening applied.

16.6 Final Pipeline State

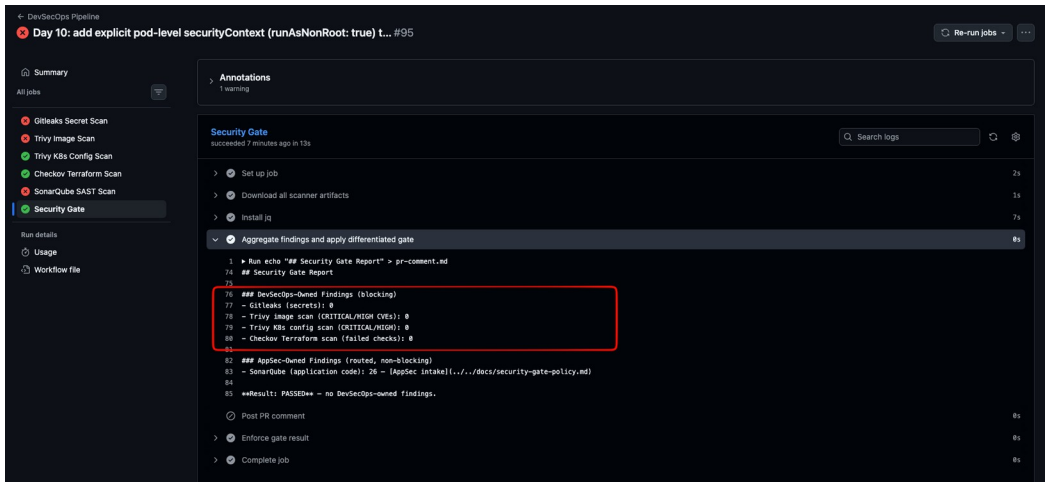


Figure A14: DevSecOps-owned pipeline stages confirmed passing following the final remediation pass.